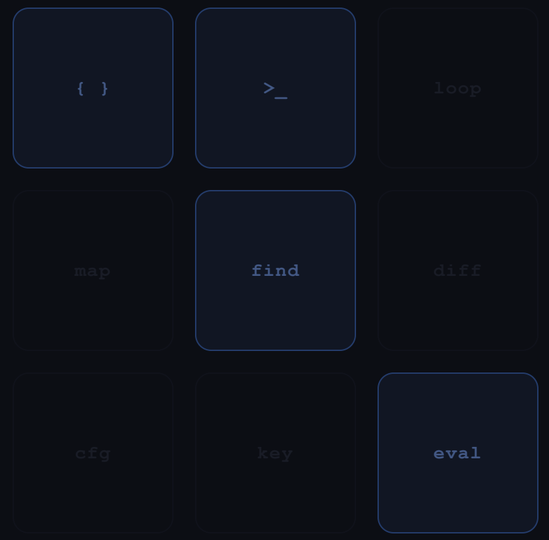
АРХИТЕКТУРА И ПРИНЦИПЫ

Agentic Harness для написания **кода**

Как LLM превращается в управляемого coding agent:
контекст, инструменты, sandbox, цикл планирования и проверка

Software Engineers / Tech Leads / AI Engineers 15 слайдов



МОТИВАЦИЯ

LLM сама по себе не меняет кодовую базу

Ей нужны: контекст, инструменты, ограничения, проверка, память и контроль ошибок.

Chat Completion

Текст в вакууме

- Не видит файловую систему
- Не запускает тесты
- Не проверяет типы
- Нет обратной связи от среды
- Не откатывает ошибочные изменения

VS

Agentic Coding System

Замкнутая система

- Читает и индексирует репозиторий
- Планирует и разбивает задачи
- Генерирует и применяет патчи
- Запускает тесты и линтеры
- Анализирует ошибки и чинит

ОПРЕДЕЛЕНИЕ

Что такое agentic harness

> Harness — управляющая обвязка вокруг модели, превращающая её в исполнителя задач.

01 Планирует, читает файлы, редактирует код

02 Запускает тесты, анализирует ошибки

03 Управляет памятью и состоянием

04 Обеспечивает безопасность и аудит

Модель — reasoning core. **Harness** — execution / control layer.

АРХИТЕКТУРА

Высокоуровневая архитектура harness

ЖИЗНЕННЫЙ ЦИКЛ

Главный цикл работы агента

Надёжность появляется из итеративного цикла, а не из одного ответа модели.

1. **Understand task**
   Разбор намерения и ограничений
2. **Build context**
   Чтение файлов, индекс, семантический поиск
3. **Plan**
   Декомпозиция и план реализации
4. **Act**
   Генерация и применение патча
5. **Observe**
   Результаты: тесты, линтер, логи
6. **Repair**
   Анализ ошибок и целевой патч
7. **Verify**
   Финальная проверка и дифф-ревью
8. **Summarize / open PR**
   Описание изменений для ревьюера

CONTROL LOOP

```
while !task.done: context = build_context(task) plan = llm.plan(context, task) for step in plan.steps: result = tool_executor.run(step) if result.failed: diagnosis = llm.analyze(result.error) repair = llm.patch(diagnosis) tool_executor.apply(repair) if validation.pass(task): task.complete() break
```

КОНТЕКСТ

Как агент понимает кодовую базу

Качество контекста часто важнее размера модели.

RF

Чтение файлов

Прямой доступ к содержимому файлов проекта, выборочное чтение диапазонов строк

RM

Repo map

Обзорная карта структуры директорий, модулей и их взаимосвязей

SS

Semantic search

Векторный поиск по смыслу, а не только по ключевым словам

DG

Dependency graph

Граф импортов и зависимостей для оценки влияния изменений

SI

Symbol index

Индекс функций, классов, типов и их сигнатур по всему проекту

DI

Recent diffs & PR context

Последние изменения, открытые PR, issue и комментарии ревьюеров

! Мусор на входе = мусор на выходе. Контекст определяет потолок качества агента.

ИНТЕРФЕЙСЫ

Инструменты агента

Инструменты должны быть явными, логируемыми и ограниченными.

RF **read_file**
Чтение содержимого файла с указанием диапазона строк

SR **search / grep**
Поиск по тексту, регулярным выражениям и семантике

ED **edit_file**
Точечное редактирование через diff/patch с валидацией конфликта

SH **run_shell**
Выполнение команд в sandbox с таймаутом и лимитами

TT **run_tests**
Запуск тестов с фильтрацией и анализом провалов

LG **inspect_logs**
Чтение stdout/stderr и стектрейсов для диагностики

PM **package_manager**
Установка, обновление и аудит зависимостей

LT **linter / typechecker**
Статический анализ, проверка типов и форматирование

Каждый вызов инструмента = событие в audit log. Агент не может выйти за пределы разрешённого набора.

Планирование и разбиение задачи

1. **Intent extraction** — извлечение намерения из запроса пользователя и контекста задачи
2. **Constraints** — выявление ограничений: стиль кода, покрытие тестами, совместимость API
3. **Affected modules** — анализ зависимостей и определение затронутых файлов
4. **Implementation plan** — пошаговый план с порядком правок и ожидаемым результатом
5. **Risk analysis** — оценка рисков каждой правки и вероятных побочных эффектов
6. **Rollback strategy** — план отката: git stash, revert, feature flag

Пример декомпозиции

«Добавить OAuth login»

- **A** Schema: migration + user model fields
- **B** Backend: OAuth routes + callbacks
- **C** Frontend: login UI + redirect flow
- **D** Tests: unit + integration
- **E** Docs: API reference + setup guide

⚠ Каждая подзадача = один атомарный коммит с проверкой

Как агент меняет код

Агент должен вносить маленькие проверяемые изменения.

1. Выбрать целевые файлы
2. Сгенерировать diff / patch
3. Проверить конфликт с текущим состоянием
4. Применить patch
5. Перечитать изменённые участки
6. Минимизировать scope изменений

→ **Маленький патч легче проверить, откатить и объяснить в PR.**

ИСПОЛНЕНИЕ

Sandbox и execution environment

PG **Изоляция команд**
Каждая команда выполняется в изолированной среде с отдельными правами

TL **Лимиты времени и ресурсов**
Таймаут на команду, лимит CPU/RAM, максимальное число итераций

DN **Запрет опасных операций**
rm -rf /, force push, удаление системных файлов заблокированы

AL **Allowlist / denylist**
Белый список разрешённых команд и чёрный список опасных паттернов

SC **Работа с секретами**
Секреты не попадают в prompt и логи, доступ через переменные среды

NW **Network policy**
Ограничение исходящих соединений: только разрешённые хосты

SANDBOX BOUNDARY

npm test	ALLOWED
git diff	ALLOWED
tsc --noEmit	ALLOWED
npm install X	ALLOWED*
rm -rf /	DENIED
git push --force	DENIED
curl evil.com	DENIED

Coding agent должен быть мощным, но не безграничным. Harness — это safety boundary.

ВАЛИДАЦИЯ

Воронка проверки результата

Harness должен не только писать код, но и доказывать, что изменение работает.

- **Форматирование** ОК
- **Lint** ОК
- **Typecheck** ОК
- **Unit tests** ОК
- **Integration tests** ОК
- **Regression** ОК

Принцип доказательства

Каждый уровень воронки отсекает класс ошибок. Если патч не проходит один уровень — он не идёт дальше.

Финальный уровень — **manual reasoning review**: человек проверяет логику и архитектурные решения.

Агент пишет код + доказывает его корректность = доверие к изменениям.

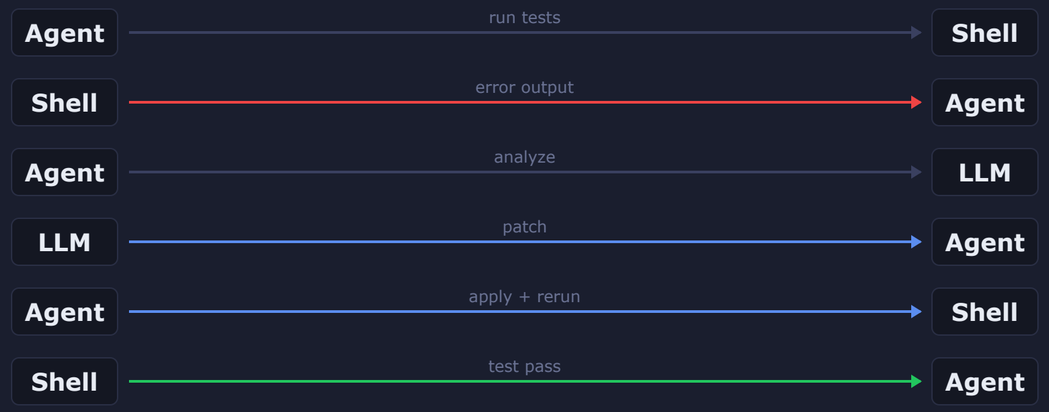
SELF-REPAIR

Ошибки и саморемонт

Типичный цикл: тест падает → анализ логов → гипотеза → патч → повторный запуск.

- X — Test failure — тест не прошёл
- ▼
- LA — Log analysis — чтение стектрейса и stderr
- ▼
- HY — Hypothesis — формулировка причины ошибки
- ▼
- PT — Targeted patch — минимальное исправление
- ▼
- OK — Rerun tests — повторная проверка

SEQUENCE DIAGRAM



GOVERNANCE

Безопасность и управление

Harness — это не только execution layer, но и safety boundary.

PG Permission gates

Каждое действие агента проходит через систему разрешений. Чтение, запись, выполнение — отдельные уровни доступа.

HA Human approval

Критичные операции требуют подтверждения: push в main, удаление файлов, изменение зависимостей, работа с продакшен-секретами.

SC Секреты и credentials

API-ключи и токены не попадают в контекст модели. Доступ через безопасное хранилище и переменные среды.

DC Destructive commands

Деструктивные команды заблокированы: force push, drop database, удаление вне рабочей директории.

SP Supply-chain risk

Аудит новых зависимостей: проверка лицензий, известных уязвимостей и репутации пакета перед установкой.

AL Audit log

Полная история действий: какие инструменты вызывались, какие файлы менялись, какие команды выполнялись.

Policy enforcement — правила безопасности не выключаются, даже если агент «уверен» в правильности действия.

МЕТРИКИ

Telemetry, evals и качество

78%
Task success rate

96%
Tests passed

3.2
Avg iterations

47
Avg patch size (lines)

12%
Human intervention

8 min
Time-to-PR

2.1%
Regression rate

$0.34
Cost per task

Зачем нужны evals

- Сравнение версий модели и конфигурации harness
- Обнаружение регрессий при обновлении инструментов
- Оптимизация баланса стоимости и качества
- Бенчмарки по типам задач: bug fix, feature, refactor

Без evals агентская система не улучшается управляемо. Метрики = обратная связь для разработчиков harness.

ИТОГИ

5 принципов хорошего coding harness

1 **Explicit tools**
Инструменты объявлены, ограничены и логируются. Нет скрытых возможностей.

2 **Tight feedback loop**
Итеративный цикл: действие → наблюдение → корректировка за секунды, не минуты.

3 **Scoped context**
Только релевантный контекст. Качество важнее объёма.

4 **Safe execution**
Sandbox, permissions, audit log. Мощность ограничена политиками безопасности.

5 **Measurable quality**
Evals, метрики, трейсинг. Улучшение только через измеримые данные.

LLM генерирует гипотезы; harness превращает их в проверяемые изменения кода.

— Главный тезис презентации